



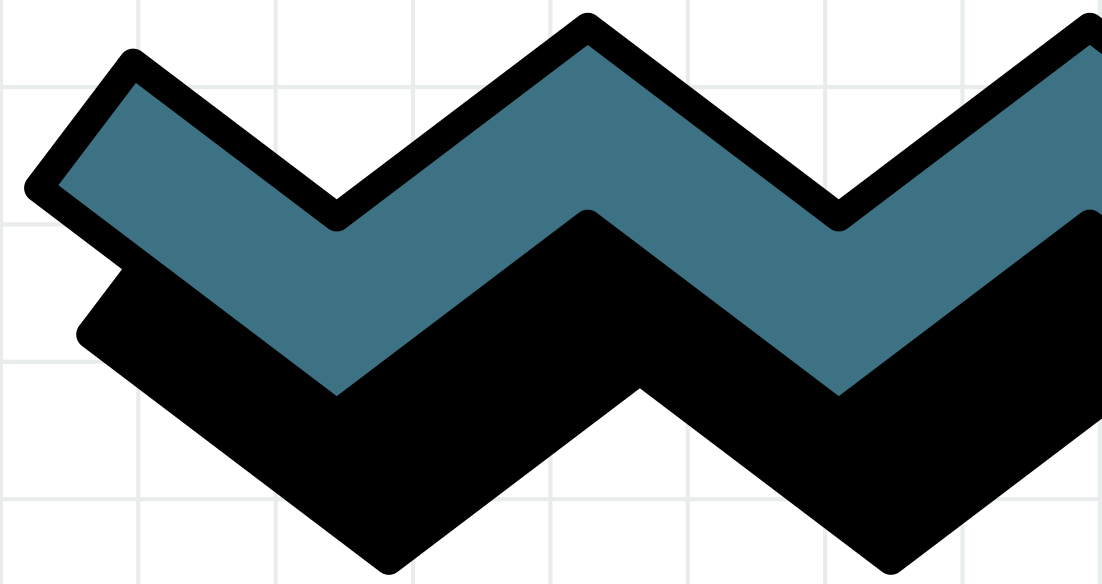
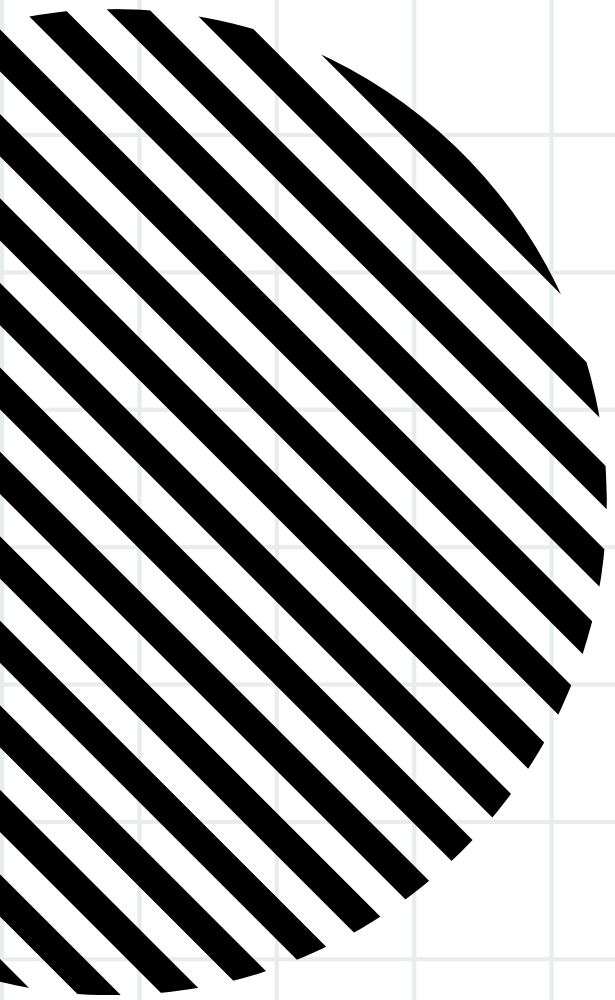
Universidad Autónoma de Campeche

Facultad de Ingeniería

Concurrencia y Paralelismo

Tecnologías de Software

5to “A”

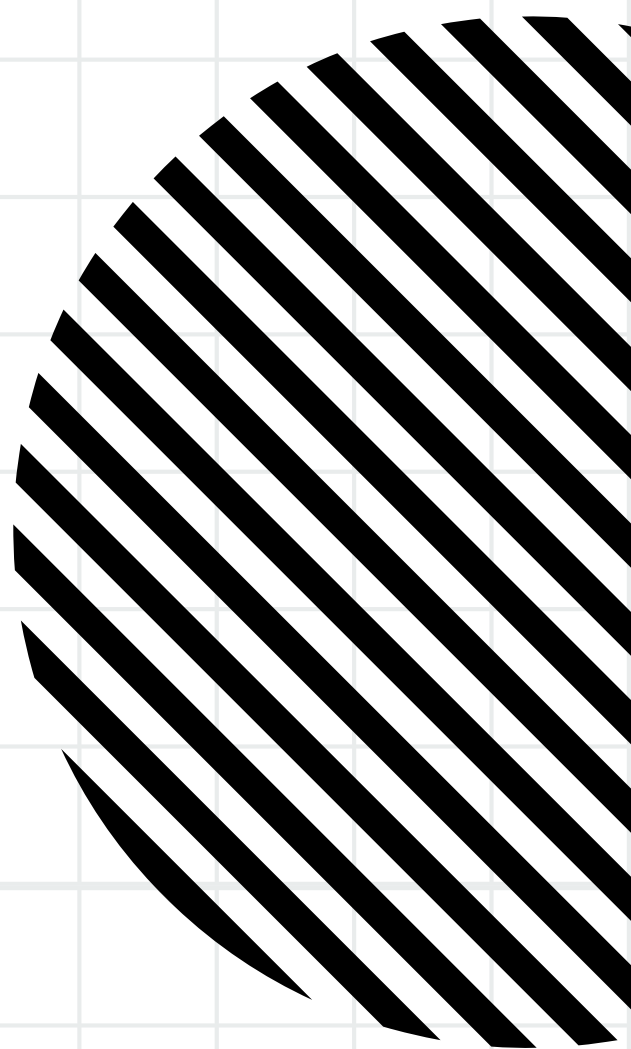
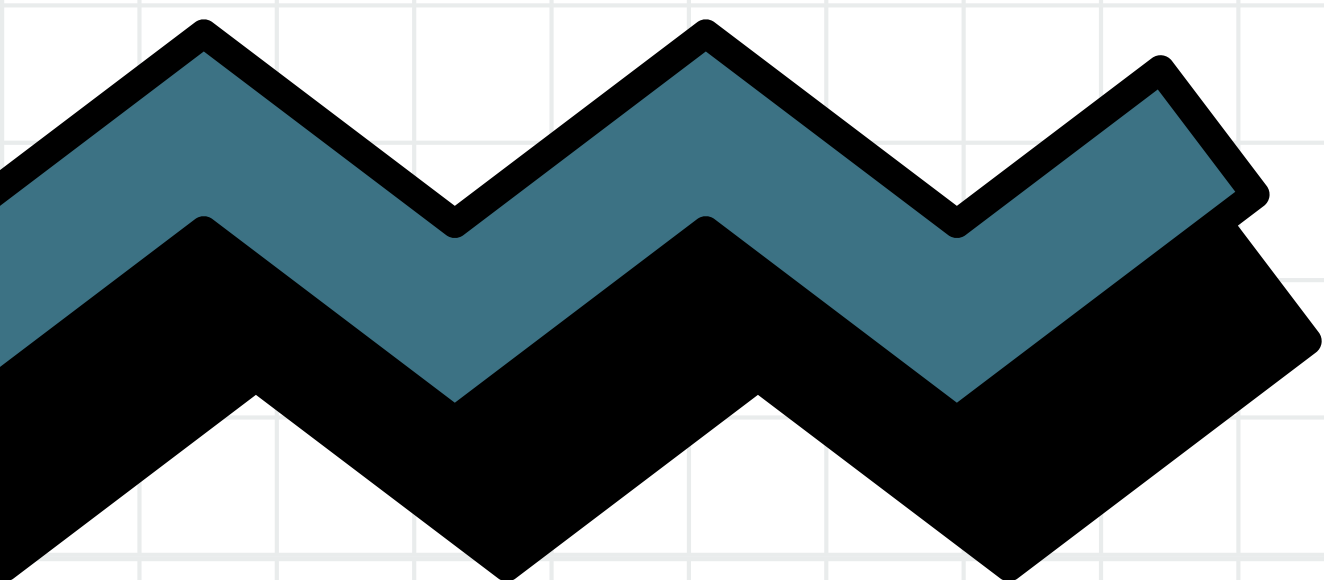


PRACTICA 01

Sección Crítica

1er parcial

28 de Agosto de 2026



Alumno: Victor Adrian Romero Minaya

Matricula: 67350

Docente: Sergio A Noh Puch



INTRODUCCIÓN

En el presente reporte se documenta el desarrollo y análisis de una práctica fundamental en la programación concurrente. El objetivo principal de la actividad es implementar un pseudocódigo en C que utiliza múltiples hilos para modificar una variable global compartida. A través de este ejercicio, se busca observar el comportamiento del sistema operativo al gestionar hilos, analizar a nivel de ensamblador por qué ocurre la pérdida de datos y, finalmente, aplicar soluciones prácticas para entender a fondo conceptos clave como las condiciones de carrera y la protección de la sección crítica.

DESARROLLO

Instrucción de la actividad: Implementar el pseudocódigo proporcionado y experimentar con diferentes mecanismos de sincronización para entender las condiciones de carrera y la sección crítica.

1. Inicio de la implementación y el primer problema: Para comenzar con esta tarea, tuve que implementar el código base en un archivo llamado `race_test.c` que hicimos en clase. El objetivo era iniciar dos hilos de ejecución simultáneos: uno encargado de sumarle 1 a la variable `global_counter` (5,000,000 veces) y otro encargado de restarle 1 (5,000,000 veces). Para visualizar el progreso y comportamiento de los hilos de forma persistente, se configuró un búfer dinámico que registra cada movimiento en un archivo de salida llamado `readme.txt`; en lugar de abrir y cerrar el archivo en cada iteración, se abre una sola vez y se dividen las impresiones cada 500,000 iteraciones, para no afectar la velocidad de ejecución del programa

2. Ejecución sin sincronización: Antes de analizar el mutex, se pidió comprobar de forma práctica qué pasa cuando se ejecuta el programa sin ningún mecanismo de sincronización. Para esto, hice una segunda versión del código llamada `race_test_sin_sync.c`, que es exactamente igual a `race_test.c` pero sin el `pthread_mutex_t` ni las líneas de `pthread_mutex_lock` y `pthread_mutex_unlock`, dejando que ambos hilos accedan a `global_counter` libremente y sin ningún control.

Matemáticamente, el resultado final debería ser 20, ya que el hilo 1 suma 1 cinco millones de veces y el hilo 2 resta 1 cinco millones de veces, por lo que ambas operaciones deberían cancelarse ($20 + 5000000 - 5000000 = 20$). Sin embargo, al correr el programa varias veces sin protección, obtuve valores como -21342 o -700281, es decir, nunca fue exactamente 20 y además el resultado cambió cada vez que ejecuté el programa.

DESARROLLO:

```
Thread1 - Contador: 21
Thread1 - Contador: 500021
Thread1 - Contador: 1000021
Thread 2 - Contador: 1155898
Thread1 - Contador: 987844
Thread 2 - Contador: 884019
Thread1 - Contador: 1123221
Thread 2 - Contador: 1005418
Thread1 - Contador: 806275
Thread 2 - Contador: 714661
Thread 2 - Contador: 333902
Thread1 - Contador: 105776
Thread 2 - Contador: 14272
Thread 2 - Contador: -92000
Thread1 - Contador: -66585
Thread 2 - Contador: -167446
Thread1 - Contador: -107274
Thread 2 - Contador: -173470
Thread1 - Contador: -135837
Thread 2 - Contador: -295934
Valor final de global_counter: -700281
```

```
Thread1 - Contador: 21
Thread1 - Contador: 500021
Thread1 - Contador: 1000021
Thread1 - Contador: 1500021
Thread 2 - Contador: 1810680
Thread1 - Contador: 1791019
Thread 2 - Contador: 1738535
Thread1 - Contador: 1759828
Thread 2 - Contador: 1667965
Thread1 - Contador: 1676014
Thread 2 - Contador: 1570950
Thread1 - Contador: 1557303
Thread 2 - Contador: 1578892
Thread1 - Contador: 1725524
Thread 2 - Contador: 1613466
Thread1 - Contador: 1752710
Thread 2 - Contador: 1858717
Thread 2 - Contador: 1457500
Thread 2 - Contador: 957500
Thread 2 - Contador: 457500
Valor final de global_counter: -42499
```

3. ¿Qué es una condición de carrera? Al analizar el comportamiento de la concurrencia, fue necesario abordar el concepto de condición de carrera que plantea la actividad. Esto sucede cuando dos o más hilos intentan acceder y modificar un mismo recurso compartido (en este caso, la variable `global_counter`) al mismo tiempo. Dado que el sistema operativo decide de forma arbitraria en qué orden se ejecutan los hilos, no hay un control de quién hace qué y cuándo. Si el hilo 1 y el hilo 2 leen la variable en el mismo milisegundo, ambos harán su operación matemática sobre el mismo número de origen. El hilo que guarde su resultado al último simplemente sobrescribirá y borrará el progreso que hizo el otro hilo. Literalmente "compiten" en una carrera por ver quién guarda el dato al final, provocando una pérdida de información :(

DESARROLLO:

4. **Análisis del código ensamblador:** Para saber por qué se interrumpen los datos a nivel de hardware, se ejecutó en la terminal el comando `gcc -S race_test.c` para generar y analizar el código ensamblador, tuve que investigar un poco acerca de como hacerlo y localizar correctamente las instrucciones. Ahí pude comprobar que una suma en C son varias líneas y no solo un proceso directo. Siguiendo las indicaciones de la actividad, busqué específicamente las instrucciones `movl`, `addl` y `subl` dentro del archivo `.s` para entender qué estaba pasando. Investigando encontré lo siguiente:

- `movl`: Esta instrucción mueve (o copia) el valor actual de la memoria RAM hacia un registro temporal dentro del procesador.
- `addl` / `subl`: Aquí se realiza la matemática pura. En el hilo de la suma se analizó la instrucción `addl` (`add long`), que sirve para sumarle 1 al registro. En el hilo de la resta, se vio la instrucción `subl` (`subtract long`) que sirve para restarle 1.
- `movl`: Finalmente, esta instrucción aparece de nuevo para mover el resultado actualizado desde el registro de vuelta a la memoria RAM.

Al ver que la computadora requiere estos tres pasos separados para una simple modificación, el problema de concurrencia me quedó mas claro, el sistema operativo puede pausar a un hilo justo después de su primer `movl`, mientras ese hilo está pausado, el otro hilo entra, modifica la variable y la guarda. Cuando el primer hilo "despierta" y hace su `addl` y su último `movl`, termina guardando un dato viejo, corrompiendo el resultado total.

```
46  .L3:
47      leaq    mutex(%rip), %rax
48      movq    %rax, %rdi
49      call    pthread_mutex_lock@PLT
50      movl    global_counter(%rip), %eax
51      addl    $1, %eax
52      movl    %eax, global_counter(%rip)
53      movl    global_counter(%rip), %edx
54      leaq    .LC1(%rip), %rsi
55      leaq    -112(%rbp), %rax
56      movl    %edx, %ecx
57      movq    %rsi, %rdx
58      movl    $100, %esi
```

```
127  .L8:
128      leaq    mutex(%rip), %rax
129      movq    %rax, %rdi
130      call    pthread_mutex_lock@PLT
131      movl    global_counter(%rip), %eax
132      subl    $1, %eax
133      movl    %eax, global_counter(%rip)
134      movl    global_counter(%rip), %edx
135      leaq    .LC4(%rip), %rsi
136      leaq    -112(%rbp), %rax
137      movl    %edx, %ecx
138      movq    %rsi, %rdx
```

DESARROLLO:

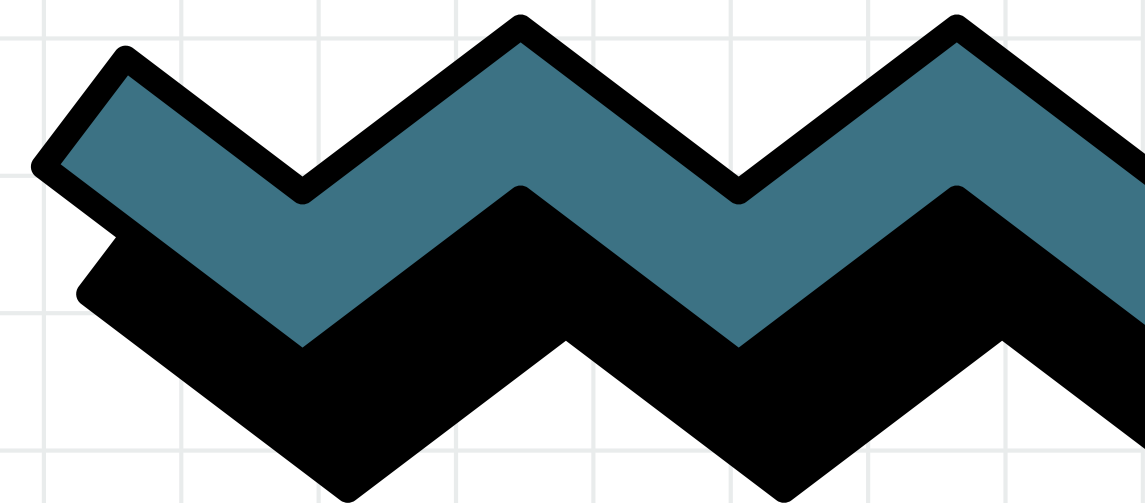
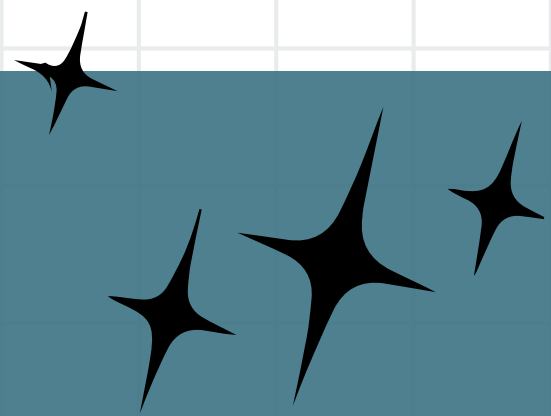
4. Entendiendo la sección crítica y aplicando sincronización: La sección crítica es la parte exacta de código donde los hilos acceden y modifican el recurso compartido (las líneas donde ocurre el incremento/decremento de `global_counter` y la escritura en el búfer según vi). Para explicar este concepto de forma sencilla (que espero entienda profe), me hace pensar en una partida de overwatch donde dos jugadores de mi equipo están a un punto de vida y corren hacia el único botiquín de curación del mapa. Ese botiquín es la "sección crítica". Si los dos tocan el botiquín en el mismo momento exacto, el código del juego tiene que decidir quién se cura. Si no hubiera un control, el servidor intentaría curar a los dos, restando "1" a un botiquín que ya no existe, lo que provocaría un error o una pérdida de datos (la condición de carrera je). El mutex pues actúa como el árbitro del servidor y detecta quién entró a la hitbox una milésima de segundo antes, cierra la puerta para que nadie más lo tome, le aplica la curación a ese jugador, y hace desaparecer el botiquín, el segundo jugador simplemente se queda sin nada porque el botiquín ya fue gestionado en orden, ni modo. Por eso vi que en la programación, se requiere proteger esta sección. En el código implementado, se utilizó el mecanismo de sincronización `pthread_mutex_t`. Al envolver la sección crítica con `pthread_mutex_lock` y `pthread_mutex_unlock`, se obliga a los hilos a respetar su turno.

Como evidencia del correcto funcionamiento de esta sincronización, pues vi que el archivo `readme.txt` muestra que los hilos no se interrumpen, manteniendo el orden de los datos sin generar una condición de carrera:)

```
Thread1 - Contador: 21
Thread 2 - Contador: 335402
Thread1 - Contador: 359537
Thread 2 - Contador: 299918
Thread1 - Contador: 263102
Thread 2 - Contador: 355216
Thread1 - Contador: 348854
Thread 2 - Contador: 347441
Thread1 - Contador: 343128
Thread 2 - Contador: 276913
Thread1 - Contador: 291867
Thread 2 - Contador: 320495
Thread1 - Contador: 440943
Thread1 - Contador: 602806
Thread 2 - Contador: 601768
Thread1 - Contador: 699226
Thread 2 - Contador: 714138
Thread1 - Contador: 766620
Thread 2 - Contador: 694094
Thread 2 - Contador: 500019
Valor final de global_counter: 20
```

```
Thread1 - Contador: 21
Thread 2 - Contador: 295859
Thread1 - Contador: 287648
Thread 2 - Contador: 369152
Thread1 - Contador: 309946
Thread 2 - Contador: 230983
Thread1 - Contador: 279738
Thread 2 - Contador: 326547
Thread1 - Contador: 204393
Thread 2 - Contador: 143833
Thread1 - Contador: 184017
Thread 2 - Contador: 151224
Thread1 - Contador: 85620
Thread 2 - Contador: 59189
Thread1 - Contador: 13706
Thread 2 - Contador: 18715
Thread1 - Contador: 67269
Thread 2 - Contador: 73654
Thread 2 - Contador: -66833
Thread1 - Contador: -115597
Valor final de global_counter: 20
```

CONCLUSION



En conclusión, esta práctica me demostró que las operaciones matemáticas aparentemente simples en lenguajes como C no se procesan de una sola vez. El análisis del código ensamblador me dejó claro cómo la interrupción entre las instrucciones `movl`, `addl` y `subl` es una causa de las condiciones de carrera cuando no hay un control de concurrencia. Experimentar con el candado lógico (mutex) comprobó que delimitar y proteger la sección crítica es un paso absolutamente indispensable; es el único mecanismo que asegura la integridad de los datos al obligar a los hilos a respetar sus turnos antes de modificar cualquier recurso compartido.



BIBLIOGRAFIA



- Evans, D. (2006). x86 Assembly Guide. University of Virginia Computer Science. Recuperado de: <https://www.cs.virginia.edu/~evans/cs216/guides/x86.html>
- IBM. (s.f.). Condiciones de carrera y sincronización de hebras. IBM Documentation. Recuperado de: <https://www.ibm.com/docs/es/aix/7.2?topic=programming-race-conditions-thread-synchronization>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Conceptos de sistemas operativos (9.ª ed.). John Wiley & Sons.



¡Gracias por leer!